Contents

SISTEMSKO PROJEKTOVANJE 2

Teorijska osnova i Attention mehanizam 2

Analiza profiler rezultata i softversko modelovanje 5

Ekstrakcija i C++ modeliranje 6

Detaljan opis implementacije 8

Virtualna platforma 10

Verifikacije modela i performanse 15

SISTEMSKO PROJEKTOVANJE

Teorijska osnova i Attention mehanizam

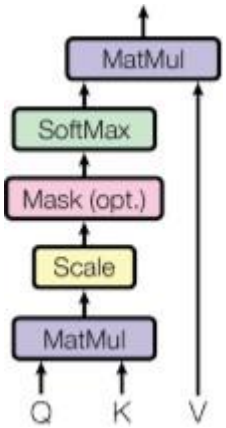
Mehanizam, koji je ključna komponenta Transformers arhitekture, igra glavnu ulogu u poboljšanju mogućnosti različitih modela (u našem slučaju whisperAI model) da se istovremeno fokusiraju na više drugačijih sekvenci ulaznih signala, posebno je veći doprinos u zadacima poput mašinskog učenja, generisanja teksta i još mnogo toga.

Ulaz se može predstaviti kao skup matrica Q (Query), K (Key), i V (Value):

- **Q –** pitanje:"na koje delove audio snimka trebam da obratim pažnju u datom trenutku"?
- **K –** predstavlja index svih segmenata zvučnog fajla. Na osnovu njih model poredi svoj upit (Q) sa dostupnim informacijama kako bi pronašao najbolje poklapanje,
- **V –** stvarni zvučni podatak. Kada se pitanje (Q) i ključ (K) poklope, model uzima vrednost iz ove matrice kako bi generisao izlaz.

Self Attention se definiše pomoću formule:

$$\text{Attention}(Q,\ K,\ V)\ =\ \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Slika 1. Arhitektura Attention mehanizma

Matrice Q, K i V izvode se iz istog izvora (izlaz prethodnog sloja). Ovo omogućava modelu da analizira odnose između različitih delova sekvence. Nakon množenja matrica Q, K i V, dobijaju se sirovi podaci na koje se primenjuje **softmax funkcija,** koja osigurava da suma svih težina bude tačno 1. Ova funkcija pretvara sirove brojeve u verovatnoće, govoreći modelu da obrati pažnju na neku specifičnu reč u rečenici.

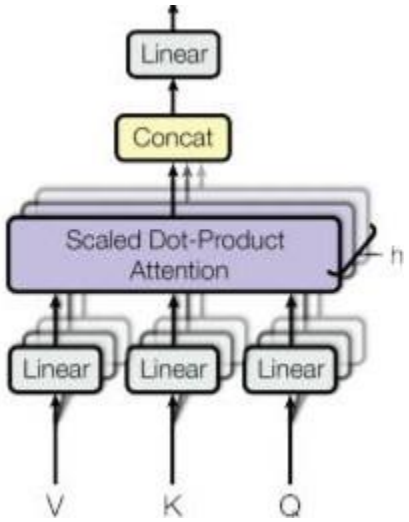
Na kraju se rezultat softmax funkcije množi sa vrednostima V, gde se na izlazu dobija **težinska suma**.

Princip rada Multi-Head Attention mehanizma

Kod mehanizma pažnje sa više glava (eng. Multi-Head Attention), ulaz se deli na više glava, što omogućava modelu da uhvati različite vrste odnosa u podacima. Umesto korišćenja samo jednog većeg skupa prethodno spomenutih matrica, ulazne reprezentacije (eng. embeddings) se projektuju u više skupova (glava), pri čemu svaka glava ima sopstvene matrice. Proces se odvija u sledećim koracima:

- Ulaz X se projektuje u više potprostora manjih dimenzija koristeći različite težinske matrice. $\mathbf{Q_i = XW^Q}$, $\mathbf{K_i = XW^K}$, $\mathbf{V_i = XW_i^V}$

  gde **i** označava index glave.
- Svaka glava nezavisno računa svoj attention koristeći formulu skalarnog proizvoda (eng. Scaled Dot-Product Attention).
- Izlaz svih pojedinačnih glava se spajaju u jednu matricu.
- Na kraju se primenjuje konačna težinska matrica kako bi se spojeni izlaz transformisao u odgovarajući vektor.



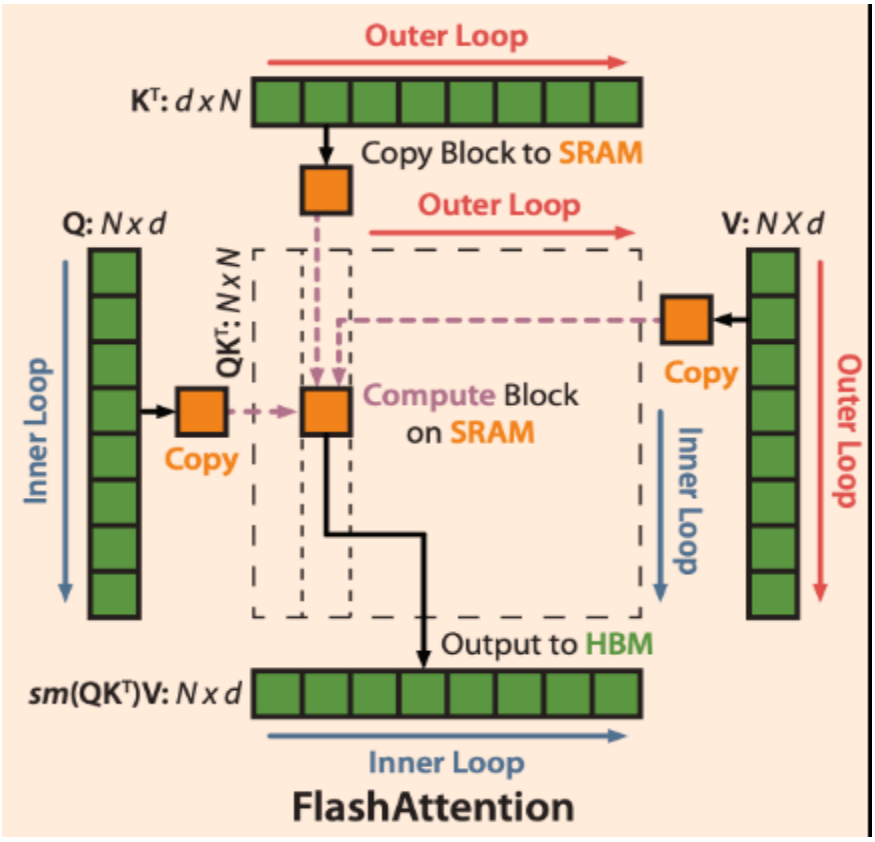
Slika 2. Arhitektura Multi-Head Attention-a

Matematički, mehanizam pažnje saviše glava se može prikazati kao MultiHead(Q, K, V) = Concat(head$_1$, head$_2$, … , head$_h$)W^O odakle sledi: head$_i$ = Attention(QW^Q, KW^K, VW^V). W^O je finalna težinska matrica koja projektuje spojeni izlaz nazad u dimenzije koje zahteva model.

Kod Multi-Head Attention-a različite glave se fokusiraju na različite aspekte ulaza. Radeći paralelno, višestruke glave omogućavaju bolje učenje zavisnosti unutar podataka. Model se ne oslanja samo na jedan obrazac, čime se smanjuje rizik od preprilagođavanja (eng. overfitting).

Flash Attention mehanizam

Flash Attention je hardverska optimizacija Attention mehanizma koja koristi tehniku deljenja podataka na manje blokove. Usled strožijih memorijskih ograničenja na FPGA ploči koju koristimo, za hardversku realizaciju je odabrana Flash Attention arhitektura umesto klasičnog Multi-Head Attention pristupa. Flash Attention optimizuje pristup memoriji i smanjuje potrebu za skladištenjem velikih međurezultata, ali pritom ne menja samu matematiku mehanizma. Ovaj mehanizam daje identične rezultate kao standardni MHA, s tim je naš originalni C++ model ostao identičan i koristi se za generisanje zlatnih vektora.



Slika 3. Šematski prikaz Flash Attention (FA) algoritma i princip tiling-a

Analiza profiler rezultata i softversko modelovanje

Pre same faze ekstrakcije i modelovanja u C++ jeziku, izvršeno je profilisanje originalnog Python koda kako bismo precizno identifikovali delovi modela koji najviše opterećuju procesor.

Analizom hijerarhije poziva, uočeno je da ukupno vreme transkripcije u testiranoj aplikaciji (final_app.py) iznosi 7.7s. Unutar tog vremena, dominantnu ulogu imaju funkcije za generisanje tokena:

- **WhisperForConditionalGeneration.generate** i
- **GenerationMixin.generate**

Ove funkcije se pozivaju u petlji za svaki novi generisani token. One zapravo služe da u svakoj iteraciji pokreću složene proračune unutar dekodera, gde se nalazi **Multi-Head attention (MHA)** mehanizam.

Da bi smo lakše razumeli šta konkretno troši vreme unutar pomenutih funkcija, korišćen je detaljan prikaz operacija.

Area Selection | Slices | Pivot Table | Slice Flamegraph

Name	Wall duration ↓	Self duration	Avg Wall duration	Occurrences
	Σ 2.641s	Σ 969.1ms		Σ 39575
model_inference	969.1ms	128.5ms	969.1ms	1
aten::linear	477.0ms	9.899ms	314.3µs	1518
aten::addmm	292.3ms	274.7ms	225.5µs	1296
aten::bmm	192.7ms	192.0ms	263.2µs	732
aten::matmul	159.1ms	2.290ms	716.5µs	222
aten::mm	155.5ms	155.3ms	700.4µs	222
aten::softmax	62.43ms	952.5µs	170.6µs	366
aten::_softmax	61.48ms	61.48ms	168.0µs	366

Slika 4. Potrebno vreme izvršavanja određenih matematičkih operacija

Iz ove tabele se jasno vidi da su bottlenecks upravo matematičke operacije koje čine srž Multi-Head Attention mehanizma:

- Operacije **aten::linear** (477 ms), **aten::matmul** (159.1 ms) i **aten::bmm** (192.7ms) zajedno čine ubedljivo najveći deo vremena. U kontekstu projekta, ove operacije direktno odgovaraju linearnim transformacijama za dobijanje Q, K i V matrica, kao i samom procesu izračunavanja težine pažnje ($Q \times K^T$).
- Operacija **aten::softmax** (zajedno sa internim varijacijama) troši preko 62ms. Ovo je ključna komponenta MHA koja osigurava da suma težina bude 1.
- Operacije poput **transpose, reshape** i **view** su neophodne za deljenje i spajanje glava (Split/Merge Heads) unutar MHA modula.

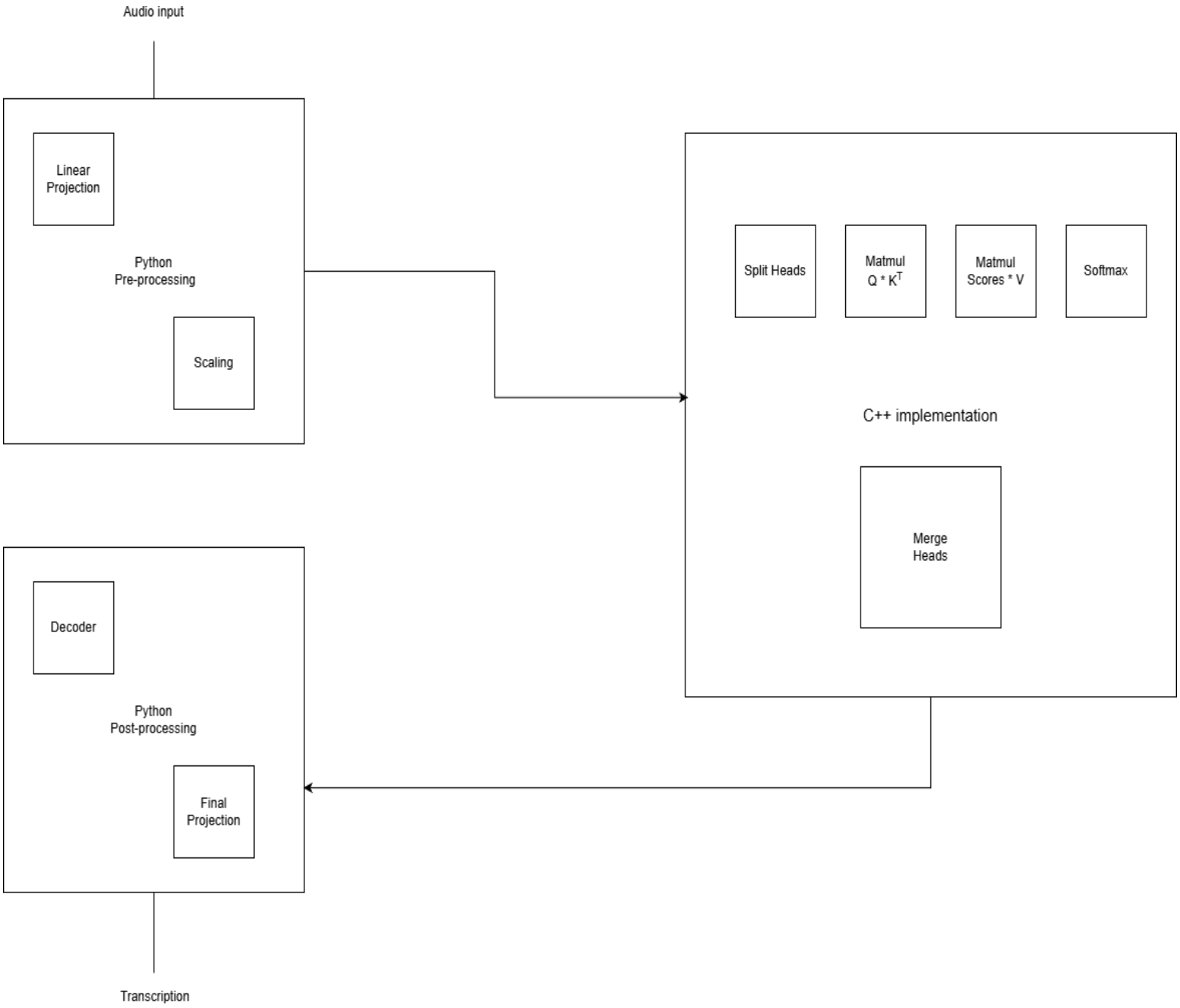
Ovi podaci nam govore Python implementacija troši vremena na matrične operacije. To nam dozvoljava da kompletan MHA mehanizam ekstrahujemo i implementiramo.

Ekstrakcija i C++ modeliranje

Prva faza projekta jeste ekstrakcija MHA iz originalnog WhisperAI modela, koji je implementiran u jeziku Python. Primarni cilj ove faze bio je prelazak sa softverskog algoritma na implementaciju u jeziku C++, koja je bolja za kasniju hardversku realizaciju. Ključni korak u verifikaciji je razvoj **pybind_wrapper.cpp** fajla. Ovaj model predstavlja direktnu implementaciju algoritma prilagođenu za kasniji prelazak na hardver, ali sa integrisanom podrškom za Python okruženje. Glavni cilj je bio osigurati da ovaj C++ kod imitira ponašanje originalnog modela. Proces verifikacije je sledeći:

- originalni Python sloj je isključen,
- umesto njega je integrisan naš **pybind_wrapper.cpp** fajl (preko Pybind11),
- izvšeno je testiranje novog modela istim audio fajlovima,

Nakon testiranja, izlaz je bio identičan izlazu originalne implementacije, i na taj način je utvrđena tačnost našeg modela. Princip povezivanja i koraci izvršavanja je prikazano pomoću blok šeme.



Slika 5. Blok dijagram final_app.py i pybind_wrapper.cpp

Detaljan opis implementacije

Nakon definisanja opšte arhitekture, implementacija je podeljena na softverski (final_app.py) i hardverski (pybind_wrapper.cpp) deo. Odnosno, konkretno se misli na dalju realizaciju hardvera koji treba da obavlja posao pybind_wrapper-a i host koji je zapravo final_app.

Softverski deo:

- Linearna projekcija – ulazi se projektuju u Q, K i V matrice koristeći predefinisane težine modela.
- Skaliranje – matrica Q se skalira direktno u Python-u kako bi se smanjila kompleksnost ulaza hardverskog modula. Primer koda: scale_factor = 1.0 / sqrt(self.head_dim) q_list = Q.detach().cpu().numpy() * scale_factor.
- Finalna projekcija – rezultati, dobijeni iz hardvera, se dekoduju i zatim se primenjuje finalna projekcija. Primer koda: attn_output = self.out_proj(attn_output).

Hardverski deo:

- Deljenje glava – ulazne matrice se dele na nezavisne glave. U našem slučaju je to osam glava.
- Množenje matrica – za svaku glavu se pojedinačno vrši množenje matrica Q i K.
- Softmax – nad prvim rezultatom monženja se primenjuje softmax.
- Drugo množenje – rezultat softmax scores se zatim množi matricom V.
- Spajanje glava – manje matrice, podeljenje na glave, se pritom spajaju u jednu veliku matricu pre finalne projekcije.

Uspešnost oba dela rezultuje u ispravnoj transkripciji.

Bitska analiza

Nakon što smo potvrdili da je naša implementacija hardvera funkcionalna, sledeća faza projekta jeste prelazak na aritmetiku u fiksnom zarezu (fixed-point) kako bismo smanjili potrošnju resursa na FPGA čipu, uz zadržavanje preciznosti.

Za svaku operaciju u MHA (npr. softmax), analizirali smo opseg vrednosti pomoću naše funkcije **analyze_bits**. Format broja je definisan kao $Q_{I,F}$ gde je:

- I (integer bits) – broj bitova za celobrojni deo, određen na osnovu formule: $I = \lceil \log_2(\text{max_abs_val}) \rceil + 1$ (1 bit je potreban za znak broja)
- F (fractional bits) – broj bitova za izlomljeni deo, određen na osnovu najmanje vrednosti koja utiče na rezultat.

Na osnovu podataka dobijenih audio signalom, primer rezultata funkcije za analizu broja bitova su:

- Analiza: Ulaz Q | Opseg: (6.89179e-08 ... 1.24322) | Potreban format: Q2.20 (Ukupno: 22 bita)
- Analiza: Head 0 Scores | Opseg: (4.19939e-06 ... 12.7354) | Potreban format: Q5.18 (Ukupno: 23 bita)
- Analiza: Head 4 Scores(Softmax) | Opseg: (1.00019e-10 ...0.988285)| Potreban format: Q2.20 (Ukupno: 22 bita)...

Ovi rezultati nam pokazuju da je opseg izlomljenog dela dosta važniji za očuvanje tačnosti modela. Ova vrsta analize je jako bitna pri prelasku sa **double** i **float** na **sc_fixed** ili **ap_fixed**. Na primer, za Ulaz Q, na osnovu analize vidimo da nam je potrebno 22 bita, 2 za celi, i 20 za izlomljeni deo. Tako da, to možemo definisati kao **using DATA_T = sc_fixed<22, 2>,** gde je od 22 bita, 2 celobrojni deo, a 20 izlomljeni deo, što spada u opseg koji je nama potreban.

Primer pozivanja funkcije u kodu:

```
Matrix scores = matmul_transpone(q_heads[h], k_heads[h]);

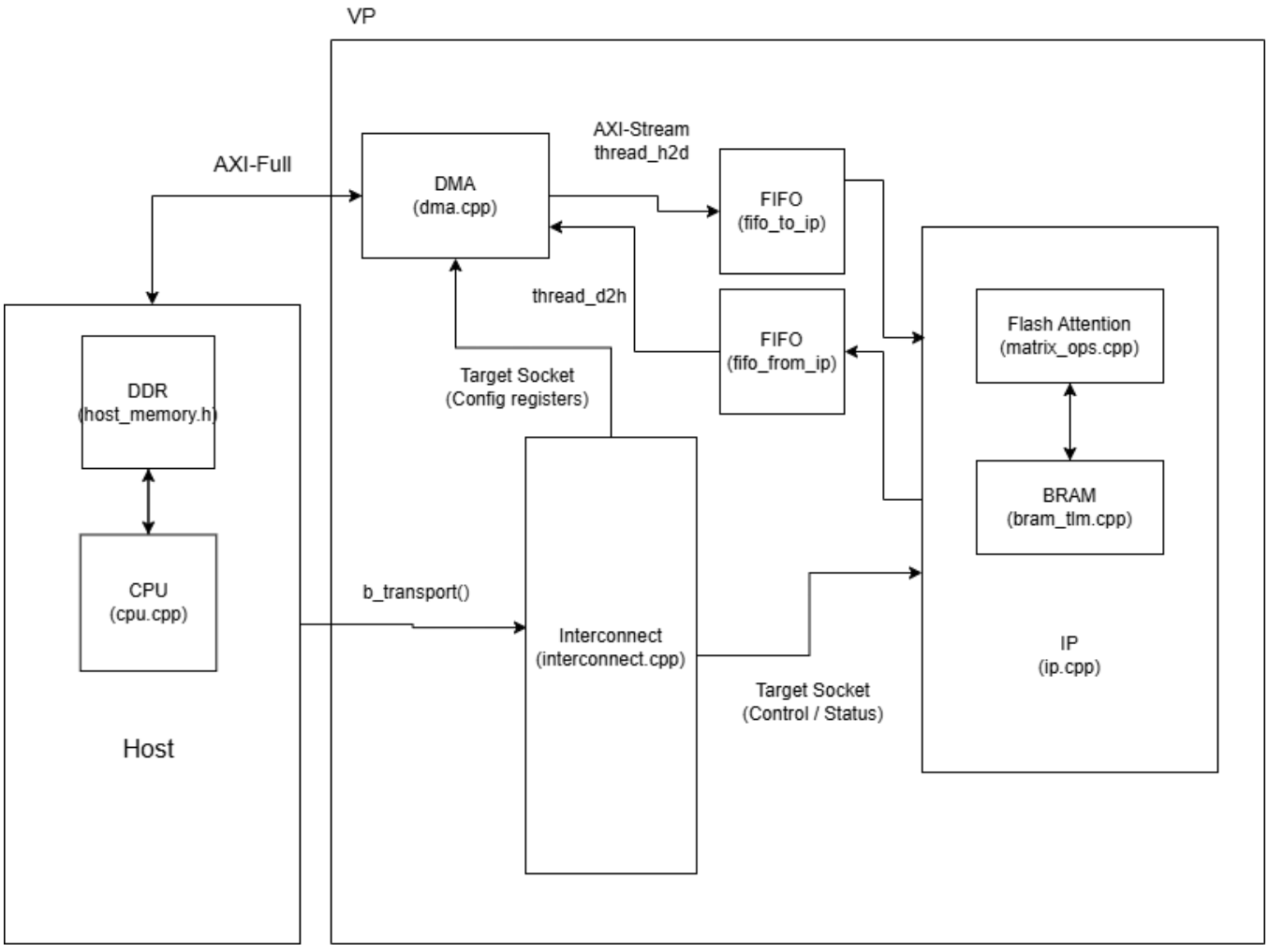
analyze_bits("string koji zelimo", scores);
```

Sve deklaracije, poput ovog primera, su definisane unutar **datatypes.h** fajla.
Kompletan prikaz bitske analize nalazi se unutar **bitska_analiza.txt** fajla.

Virtualna platforma

Nakon što smo uspešno odredili koju funkciju spuštamo na ploču, možemo razviti virtuelno okruženje kao **TLM (Transaction Level Modeling)** komponentu. Dizajnirana je tako da simulira realno hardversko okruženje pri same implementacije na FPGA čipu. Ključne komponente platforme su: model koji simulira CPU za slanje podataka preko magistrale (interconnect), host memorija (DDR) gde se nalaze težine i gde se upisuje rezultujuća matrica Y, DMA za strimovanje podataka, BRAM za skladištenje pločica (eng. tiles). Takođe, modelovali smo i magistralu koja povezuje CPU, DMA i IP.

Platforma koristi **Loosely Timed (LT)** mehanizam sa eksplicitnim modelovanjem latency pristupa memoriji (10ns po transakciji), što omogućava grubu procenu performansi sistema. Prednost ovakvog dizajna jeste da smo mogli da zamenimo MHA sa FA bez ikakve promene u interfejsu. Dovoljno je samo da zamenimo IP blok.



Slika 6. Blok šema komunikacije između host-a i VP-a

Opis komponenti virtuelne platforme i njihove funkcionalnosti:

- **CPU –** Modeluje host procesor koji pokreće softverski deo aplikacije. Učitava ulazne matrice iz fajlova, slaže ih u jedinstven stream u tačnom redosledu koji IP očekuje, upisuje ih u host RAM, konfiguriše DMA registre (adresa, dužina, source i destinacija), startuje DMA i IP preko MMIO upisa, i zatim čeka status registar dok se posao ne završi.
- **INTERCONNECT –** Modeluje sistemsku magistralu. Prima transakcije od CPU-a na jedan target socket i, na osnovu adrese u transakciji, rutira ih ka odgovarajućem master socket-u. Svaki periferni uređaj ima svoj opseg adresa i prosleđuje se greška ako adresa ne pripada nijednom uređaju.
- **DMA –** Modeluje DMA kontroler koji prenosi podatke između host RAM-a i IP-a kroz FIFO/stream interfejs. Ima registre i dva odvojena SC_THREAD procesa, host to device i device to host, preko kojih strimuje podatke ka IP-u koji čita iz FIFO-a i podatke iz IP-a strimuje nazad ka host-u.
- **IP -** Model koji implementira Flash Attention. Mehanizam koji HLS sintetiše u hardveru. Čita ulazni stream (Q, K, V, W i bias), obavlja flash attention algoritam (online softmax po tile-ovima) i finalnu projekciju, piše rezultat (Y) na izlazni stream. Koristi lokalnu BRAM preko DMI (Direct Memory Interface) pristupa za brzo skladištenje privremenih rezultata (tiles, O_acc, m, l) čime se simulira on-chip memorija koju bi IP imao na FPGA ploči.
- **BRAM memorija –** Modeluje on-chip BRAM koji IP koristi kao memoriju za skladištenje tile-ova i privremenih podataka (u zavisnosti od samog kapaciteta BRAM memorije). Implementira TLM target socket za read/write transakcije i nudi DMI pristup IP-u za brže simulacione pristupe bez prolaska kroz potpunu TLM transakciju (block transport) svaki put. Veličina i raspored offset-a matrica su definisani prema rasporedu podataka koje Flash Attention zahteva.
- **Host memorija (DDR) –** Modeluje glavnu DDR memoriju sistema gde CPU smešta ulazne matrice pre transfera i gde DMA upisuje finalni rezultat. Jednostavan je unsigned char (niz bajtova) sa read/write metodama.
- **FIFO bufferi (fifo_to_ip, fifo_from_ip) –** AXI-Stream interfejs između DMA i IP-a. DMA upisuje float vrednosti u fifo_to_ip, IP ih čita, zatim upisuje rezultate u fifo_from_ip. DMA ih prenosi u DDR.

Opis jednog kompletnog prolaska kroz sistem počinje u cpu::run(). CPU učitava ulazne matrice iz fajlova, slaže ih u jedinstveni niz (build_feed) u redosledu koji odgovara protokolu koji IP očekuje (bias – po Q-tile-u – po glavi: Q – K/V tile-ovi – W redovi), i upisuje ih u HostMemory preko TLM transakcije. Zatim konfiguriše DMA registre (izvorna adresa/dužina, odredišna adresa/dužina) MMIO transakcijama kroz Interconnect, koji rutira transakcije na osnovu adresa ka dma::cfg_b_transport. CPU startuje DMA i IP istovremeno (jedna MMIO transakcija), pa čeka statusne registre dok oba ne signalizuju završetak.
Paralelno, dma::thread_h2d() čita podatke iz HostMemory i upisuje ih u fifo_to_ip, odakle ih ip::run() čita kroz stream_in. IP izvršava Flash Attention algoritam koristeći BRAM memoriju (preko DMI pristupa) i piše rezultat u stream_out. dma::thread_d2h() prenosi rezultat nazad u DDR. Kada CPU detektuje da su oba flag-a gotova, čita rezultat i čuva ga u fajl, kojem zatim mi pristupamo kako bismo uporedili sa zlatnim vektorima i proverili tačnost.

Sinhronizacija i kašnjenje

Nakon svakog ciklusa izvršavanja potrebno je na neki način pratiti i ažurirati latency kako bi simulirali što bliže stvarnom hardveru. Za sinhronizaciju i tajming korišćen je TLM-2.0 Loosely-Timed (LT) stil modelovanja sa block transport-om (b_transport), a ne Approximately-Timed (AT). Tako je donešena odluka, s obzirom da je cilj brza funkcionalna simulacija pre prelaska na HLS/RTL gde tajming postaje precizan i bitan, bez odstupanja. Sinhronizacija CPU-a sa DMA/IP je polling-bazirana (čitanje status registra u petlji), ne interrupt-driven.

U testbench_vp.cpp postavljen je i globalni TLM kvantum (tlm_global_quantum::instance().set()) mehanizam temporal decoupling koji bi, uz upotrebu tlm_quantumkeeper, omogućio procesima da lokalno akumuliraju vreme i sinhrognizuju se sa simulacionim kernelom samo periodično, umesto na svaki pojedinačni wait(). Ovo dodatno ubrzava LT simulaciju kod modela sa mnogo malih transakcija. U finalnoj verziji projekta je ovo izbačeno jer se podaci strimuju, ili se koristi DMI, pa razlika u kašnjenju simulacije nije bila toliko značajna, ali je vredno napomenuti da je u stilu LT modelovanja i da su quantumkeeper metode ranije bile korišćene.

Socketi i povezivanje unutar VP

Svaka komunikacija u VP-u ide kroz par initiator/target socket-a (master/slave) iz tlm_utils biblioteke. Kako bismo uopšte testirali ispravnost IP modula, kao i same komunikacije, potrebno je da tačno povežemo module unutar vp.cpp i vp.h, kao što je već objašnjeno:

- simple_initiator_socket – koristi ga CPU (ka interconnect-u), Interconnect (ka IP-u i DMA) i IP (ka BRAM-u). Inicijator direktno poziva b_transport na vezanom target socket-u.
- simple_target_socket – koristi ga IP, DMA, i BRAM da prime transakcije. Registruju callback funkciju koja se izvršava kad neki inicijator pozove transakciju.
- multi_passthrough_target_socket – koristi ga Interconnect, jer on prima transakcije od više inicijatora na jedan target socket. Svaka veza dobija id, pa b_transport(int id, payload, delay) može da razlikuje ko je poslao transakciju ako je potrebno (u našem slučaju CPU je jedini inicijator).

Bind veze su statički definisane u konstruktoru vp klase (cpu.socket.bind(bus.target_socket)).

Alternativna arhitektura preko direktnog AXI-Full pristupa memoriji

Trenutni VP model prenosi sve podatke kroz AXI-Stream (FIFO) interfejs koji DMA puni iz HostMemory. Ovo zahteva da CPU unapred izgradi kompletan, uređen niz (build_feed) sa svim podacima u tačnom redosledu u kom ih IP očekuje, uključujući slučajeve gde se isti podaci (npr. K/V tile-ovi) ponovo upisuju u stream za svaku glavu, jer Flash Attention algoritam zahteva da se kroz K/V tile-ove prolazi više puta.

Alternativni pristup jeste da IP umesto stream porta ima sopstveni AXI-Full master port (m_axi) koji mu omogućava direktan, nasumičan pristup host memoriji. U tom slučaju IP sam generiše adrese i šalje read/write transakcije tačno za tile koji mu treba u datom trenutku, bez potrebe da CPU prethodno pripremi sve podatke u jedan veliki niz ni da se isti podaci kopiraju više puta u stream. Ovo bi smanjilo potrošnju host RAM-a i izbeglo kopiranje iste matrice podeljene na tile-ove, ali bi prebacio deo kompleksnosti u IP

Adresna mapa sistema (memory map)

Konkretna tabela na osnovu interconnect.h:

Periferija	Bazna adresa	Veličina	Registri
IP (flash attention)	0x40000000	0x100	CTRL (0x00), STATUS (0x04)
DMA	0x40001000	0x100	SRC_ADDR (0x00), SRC_LEN (0x08), DST_ADDR (0x10), DST_LEN (0x18), START (0x20), STATUS (0x24)

Interconnect::b_transport rutira na osnovu globalne adrese transakcije, ako adresa pada u opseg [IP_BASE, IP_BASE + IP_SIZE), prosleđuje ka IP-u (sa lokalno preračunatom adresom, global_addr – IP_BASE), analogno za DMA.

Pošto IP nema svoje softverske nizove za radne podatke, sve privremene strukture postoje u BramTLM na fiksnim offset-ima:

Niz	Offset (izraz)	Sadržaj
Q	0	trenutni Q tile
W	offset_Q + B × HEAD_DIM × 4	trenutni W tile
Bias	offset_W + COLS × 4	bias vektor
K	offset_bias + COLS × 4	trenutni K tile
V	offset_K + BC × HEAD_DIM × 4	trenutni V tile
O_acc	offset_V + BC × HEAD_DIM × 4	akumulator izlaza
m, l	redom posle O_acc	online-softmax statistike
Y	naa kraju, gde se upisuje rezultat	izlazni tile

Bitno je napomenuti da je ovaj raspored isti princip organizacije kao kod BRAM u HLS sintezi IP modula. Modul ip_hls.cpp ima identične logičke nizove, samo kao statičke nizove koje Vitis HLS sam mapira u BRAM blokove. Pomoću virtualne platforme modelovana je ista memorijska raspodela kako bi prelazak na HLS bio direktan, bez promene u načinu razmišljanja o memoriji.

Verifikacije modela i performanse

Verifikacija je urađena kroz tri nezavisne implementacije istog algoritma. Ako se sve tri solucije slažu, to potvrđuje da je implementacija C++ akumulatora, kao i VP tačna. Sledeće tri implementacije koje moramo proveriti su:

- PyTorch/Whisper referenca – final_app.py zamenjuje attention sloj pravog, već treniranog Whisper encoder modela (WhisperAttention) sopstvenom klasom koja presreće Q, K, V, W_out i bias tenzore pre nego što PyTorch izračuna attention, i snima ih u data/matrice/.txt. Ulazni podaci nisu nasumični, već stvarne težine iz treniranog modela, što znači da test pokriva realan opseg vrednosti, a ne proizvoljne brojeve. Transkripcija glavnog programa je identična, kao i rečenice izgovorene u datom audio fajlu koji se sempluje, u suprotnom bi značilo da je C++ implementacija pogrešna.
- C++ referenca – pybind_wrapper.cpp ili attention_module_multiple_heads.cpp predstavljaju implementaciju samog Multi-Head Attention algoritma. Radi tako što računa attention direktno iz istih Q/K/V/W/bias fajlova. Ako su rezultati indentični, to potvrđuje da je algoritam (matematika) tačan, nezavisno od bilo kakvog hardverskog modelovanja.
- SystemC TLM VP- ip.cpp, zajedno sa matrix_ops.cpp, predstavlja Flash Attention sa tiled online-softmax-om, baš onako kako mora da radi u hardveru. Flash Attention generiše iste rezultate na izlazu, kao i MHA, pa nije potrebno menjati C++ reference.

Razlog prelaska na FA je baš usled tih memorijskih ograničenja, što omogućava da zapravo spustimo čitav IP na ploču. MHA očekuje potpune matrice prilikom računanja softmax-a, dok se FA može implementirati tiling tehnikom. Tiling tehnika je jako bitna u ovom projektu, jer nije moguće učitati kompletne matrice na ploči poput Zybo Z7 10 ili 20.

Kako ne bi bilo potrebe za ručnim poređenjem rezultata, koristi se compare.py skripta. Učitava dva fajla red po red i zatim se porede rezultati. Tolerancija od 0.01 je izabrana zato što je VP implementiran u float preciznosti. Kada se kompajlira VP, rezultati se porede sa C++ referencom, za koju smo prethodno potvrdili da je transakcija ispravna, i ako su indentični, uspešno je implementiran naš IP.

Analiza performansi

Prilikom analize performansi prvo moramo znati po kom principu se odvija prenos podataka i obrađivanje istih. Potrebno je znati tačan broj ciklusa izvršavanja IP-a kako bi se odredio bottleneck u sistemu. Zbog toga je korišćen Vitis HLS alat za sintezu HLS koda i procene performansi. Po HLS izveštaju: DMA mora da dostavi Q, K i V tile kako bi IP mogao započeti izvršavanje. Sam proces izvlačenja podataka iz FIFO buffer-a je uračunat unutar konačnog broja ciklusa, tako da taj deo nije izostavljen. Broj ciklusa potrebnih za prenos datih podataka se računa na sledeći način (streamuje se sekvencijalno): Q tile + K tile + V tile = $32 \times 64 + 64 \times 64 \times 2 = 10240$ float-ova. U našem slučaju, strimujemo 32 bita po taktu, tako da je potrebno $10240 \times 10\text{ns} = 102.4\ \mu\text{s}$ da IP krene sa prvim izvršavanjem glavnog algoritma (flash_attn_step). IP-u je potrebno 50178 ciklusa da obradi samo jedan par K i V tile-ova, dok je DMA za to vreme već strimovao nove podatke u FIFO buffer. Flash_attn_step se poziva 9024 (47 Q tile × 8 glava × 24 K/V tile-ova) puta, što ga čini bottleneck-om sistema i samo kašnjenje strimovanja se može zanemariti. Na osnovu ovih analiza možemo zaključiti da je validno strimovati 4 bajta po taktu, čak iako magistrala dozvoljava više, ali po ceni resursa, što je kod nas skroz bespotrebno, jer IP sam po sebi ne može obraditi podatke dovoljnom brzinom da bi sustigao DMA.

U sledećim tabelama su prikazane performanse IP-a, kao i tačan broj ciklusa po delovima:

Deo	Broj ciklusa	Sekundi	Procenat
flash_attn_step	452,806,272	4.528	66.9%
Finalna projekcija	111,199,744	1.112	16.4%
Čitanje ulaza	72,960,512	0.730	10.8%
Overhead	38,734,173	0.387	5.7%
Pisanje izlaza	1,538,048	0.015	0.2%
Ukupno	677,238,749	6.772	100%

Modul	Slack	BRAM	DSP	FF	LUT	URAM
mha_ip	0.09	95 (79%)	40 (50%)	6824 (19%)	15739 (89%)	-

Zaključak

U okviru ovog projekta uspešno smo realizovali kompletan sistem, ekstrahovali sloj, modelovali naš u C++ jeziku, kao i implementacija virtualne platforme i optimizacija IP-a.

Odluka da se Multi-Head Attention profiliše i implementira, kao kritična funkcija koju treba spustiti na ploču, nas je motivisala da istražimo, kako o softverskoj, tako i o hardverskoj implementaciji. Sama softverska implementacija zahteva znanje iz programiranja, kao i razumevanje samog algoritma usled kompleksnih operacija. Simulacija kompletnog hardverskog okruženja za takav algoritam je izazov koji obuhvata znanje iz više oblasti, što ga i čini zanimljivim, kao i odakle ta odluka da se baš ovaj algoritam simulira, i u budućnosti razvije u stvarnom hardverskom okruženju.

Prelazak sa standardnog MHA na Flash Attention arhitekturu bio je neophodan, ali je i omogućio da se istraži nešto novo i nauče nove tehnike optimizacije, poput tiling tehnike koje će se koristiti u nekim daljim projektima usled dodatne optimizacije ili funkcionalnosti samog algoritma.

Kao sledeći korak planirana je stvarna sinteza i implementacija na FPGA ploči, kao i razvoj drajvera koji bi omogućio komunikaciju između procesora i IP jezgra. Virtualna platforma direktno služi kao osnova za razvoj drajvera, jer modeluje registarske interfejse, adresnu mapu i protokol komunikacije koji će biti prisutan na stvarnom hardveru.

Literatura

[1] Materijali sa katedre za elektroniku iz predmeta: Projektovanje elektronskih uređaja na sistemskom nivou.

[2] Multi-Head Attention: **Multi-Head Attention Mechanism - GeeksforGeeks**

[3] Flash Attention: **Dao-AILab/flash-attention: Fast and memory-efficient exact attention**

[4] Optimizacija pomoću HLS pragmi AMD: **HLS Pragmas • Vitis High-Level Synthesis User Guide (UG1399) • AMD Technical Information Portal**

[5] WhisperAI od OpenAI: **Introducing Whisper | OpenAI**